

Operating Systems: Assignment-10

Name: Suraj Shah

Admission Number: I24AI012

Q 1. To Write a C program to simulate producer-consumer problem using semaphores.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
sem_t mutex;

void* producer(void* arg) {
    int item;
    for(int i = 0; i < 10; i++) {
        item = rand() % 100;

        sem_wait(&empty);
        sem_wait(&mutex);

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&full);

        sleep(1);
    }
}
```

```

    }
    return NULL;
}

void* consumer(void* arg) {
    int item;
    for(int i = 0; i < 10; i++) {

        sem_wait(&full);
        sem_wait(&mutex);

        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&empty);

        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t p_thread, c_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&p_thread, NULL, producer, NULL);
    pthread_create(&c_thread, NULL, consumer, NULL);

    pthread_join(p_thread, NULL);
    pthread_join(c_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    return 0;
}

```

Output:

```
ubuntu@ubuntu:~$ ./pc
Produced: 83
Consumed: 83
Produced: 86
Consumed: 86
Produced: 77
Produced: 15
Consumed: 77
Produced: 93
Produced: 35
Consumed: 15
Produced: 86
Produced: 92
Consumed: 93
Produced: 49
Produced: 21
Consumed: 35
Consumed: 86
Consumed: 92
Consumed: 49
Consumed: 21
```

Q 2. Write a C program to simulate the concept of Dining-Philosophers problem.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

#define THINKING 0
#define HUNGRY 1
#define EATING 2

int state[N];
int phil[N] = {0, 1, 2, 3, 4};

sem_t mutex;
sem_t S[N];

int LEFT(int i) {
    return (i + N - 1) % N;
}

int RIGHT(int i) {
    return (i + 1) % N;
}

void test(int i) {
    if (state[i] == HUNGRY &&
        state[LEFT(i)] != EATING &&
        state[RIGHT(i)] != EATING) {

        state[i] = EATING;
        printf("Philosopher %d takes chopsticks %d and %d\n", i, LEFT(i), i);
        printf("Philosopher %d is Eating\n", i);

        sem_post(&S[i]);
    }
}

void take_fork(int i) {
```

```

sem_wait(&mutex);

state[i] = HUNGRY;
printf("Philosopher %d is Hungry\n", i);

test(i);

sem_post(&mutex);

sem_wait(&S[i]);
sleep(1);
}

void put_fork(int i) {

    sem_wait(&mutex);

    state[i] = THINKING;
    printf("Philosopher %d putting chopsticks %d and %d down\n", i, LEFT(i), i);
    printf("Philosopher %d is Thinking\n", i);

    test(LEFT(i));
    test(RIGHT(i));

    sem_post(&mutex);
}

void* philosopher(void* num) {

    int i = *(int*)num;

    while (1) {
        sleep(1);
        take_fork(i);
        sleep(2);
        put_fork(i);
    }
}

int main() {

```

```

pthread_t thread_id[N];

sem_init(&mutex, 0, 1);

for (int i = 0; i < N; i++)
    sem_init(&S[i], 0, 0);

for (int i = 0; i < N; i++)
    pthread_create(&thread_id[i], NULL, philosopher, &phil[i]);

for (int i = 0; i < N; i++)
    pthread_join(thread_id[i], NULL);

return 0;
}

```

Output:

```

ubuntu@ubuntu:~$ ./dining
Philosopher 1 is Hungry
Philosopher 1 takes chopsticks 0 and 1
Philosopher 1 is Eating
Philosopher 0 is Hungry
Philosopher 3 is Hungry
Philosopher 3 takes chopsticks 2 and 3
Philosopher 3 is Eating
Philosopher 2 is Hungry
Philosopher 4 is Hungry
Philosopher 1 putting chopsticks 0 and 1 down
Philosopher 1 is Thinking
Philosopher 0 takes chopsticks 4 and 0
Philosopher 0 is Eating
Philosopher 1 is Hungry
Philosopher 3 putting chopsticks 2 and 3 down
Philosopher 3 is Thinking
Philosopher 2 takes chopsticks 1 and 2
Philosopher 2 is Eating
Philosopher 3 is Hungry
Philosopher 0 putting chopsticks 4 and 0 down
Philosopher 0 is Thinking
Philosopher 4 takes chopsticks 3 and 4
Philosopher 4 is Eating
Philosopher 0 is Hungry
Philosopher 2 putting chopsticks 1 and 2 down
Philosopher 2 is Thinking
Philosopher 1 takes chopsticks 0 and 1
Philosopher 1 is Eating
Philosopher 2 is Hungry

```

Q 3. Write a C program to simulate the concept of Readers-Writers problem.

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t mutex, wrt;
int readcount = 0;
int data = 0;

void* reader(void* arg) {
    int id = *(int*)arg;

    while(1) {
        sem_wait(&mutex);
        readcount++;

        if(readcount == 1)
            sem_wait(&wrt);

        sem_post(&mutex);

        printf("Reader %d is reading data = %d\n", id, data);
        sleep(1);

        sem_wait(&mutex);
        readcount--;

        if(readcount == 0)
            sem_post(&wrt);

        sem_post(&mutex);

        sleep(1);
    }
}

void* writer(void* arg) {
    int id = *(int*)arg;

    while(1) {
```

```

    sem_wait(&wrt);

    data++;
    printf("Writer %d is writing data = %d\n", id, data);
    sleep(2);

    sem_post(&wrt);

    sleep(2);
}
}

int main() {
    pthread_t r[3], w[2];

    sem_init(&mutex, 0, 1);
    sem_init(&wrt, 0, 1);

    int r_id[3] = {1,2,3};
    int w_id[2] = {1,2};

    for(int i = 0; i < 3; i++)
        pthread_create(&r[i], NULL, reader, &r_id[i]);

    for(int i = 0; i < 2; i++)
        pthread_create(&w[i], NULL, writer, &w_id[i]);

    for(int i = 0; i < 3; i++)
        pthread_join(r[i], NULL);

    for(int i = 0; i < 2; i++)
        pthread_join(w[i], NULL);

    return 0;
}

```

Output:

```
ubuntu@ubuntu:~$ ./rw
Reader 1 is reading data = 0
Reader 2 is reading data = 0
Reader 3 is reading data = 0
Writer 1 is writing data = 1
Writer 2 is writing data = 2
Reader 2 is reading data = 2
Reader 1 is reading data = 2
Reader 3 is reading data = 2
Writer 1 is writing data = 3
Reader 1 is reading data = 3
Reader 3 is reading data = 3
Reader 2 is reading data = 3
Writer 2 is writing data = 4
Writer 1 is writing data = 5
Reader 1 is reading data = 5
Reader 2 is reading data = 5
Reader 3 is reading data = 5
Writer 2 is writing data = 6
Reader 2 is reading data = 6
Reader 1 is reading data = 6
Reader 3 is reading data = 6
Writer 1 is writing data = 7
Reader 3 is reading data = 7
Reader 1 is reading data = 7
Reader 2 is reading data = 7
Writer 2 is writing data = 8
Reader 1 is reading data = 8
Reader 3 is reading data = 8
Reader 2 is reading data = 8
Writer 1 is writing data = 9
```